

1 Microcontrollers

Om dit vak te kunnen volgen is *strikt gezien* geen speciale voorkennis vereist. Eerdere ervaring met microcontrollersystemen wordt uiteraard aanzien als een pluspunt.

Toch wordt het sterk aanbevolen om al over een aantal basisvaardigheden en kennis te beschikken om de leerstof vlot te kunnen volgen:

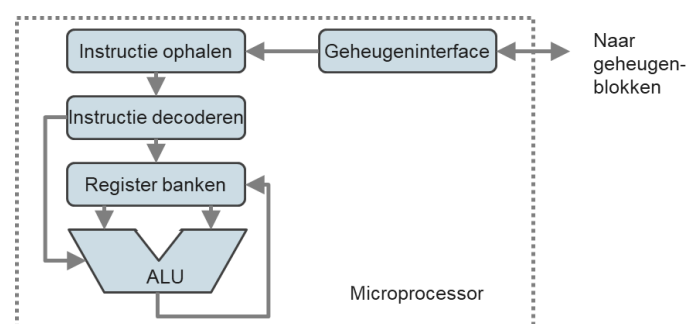
- **Algemene programmeerkennis:** het hebben van een basisinzicht in programmeren is belangrijk. Ervaring met de C-taal die we zullen gebruiken is geen vereiste, maar wordt wel als een pluspunt beschouwd.
De nodige uitleg over deze taal zal sowieso gegeven worden op het moment dat dit vereist is.
- **Kennis van digitale basislogica:** begrip van logische bouwstenen zoals AND-, OR-, en NOT-poorten. Daarnaast kan kennis van combinatorische en sequentiële logica, zoals flipflops en tellers, nuttig zijn.
- **Binaire rekenen:** het kunnen werken met binaire getallen en het begrijpen van de basisbewerkingen zoals binaire optellen, aftrekken, verschuiven ("shiften"), ... is essentieel. Ook vlot kunnen omrekenen van het ene talstelsel naar het andere zoals decimaal, binaire en hexadecimaal talstel.
Dit zal regelmatig aan bod komen tijdens het semester.

Je zal merken dat het werken met microcontrollers de max zijn, ze laten je vaak toe een wild idee om te zetten naar een werkend geheel!

1.1 Definitie

Wat is nu het verschil tussen een microcontroller of microprocessor? Velen kunnen deze vraag niet onmiddellijk beantwoorden, hoewel dit niet zo moeilijk is.

Een **microprocessor** (soms ook **CPU** genaamd, staat voor Central Processing Unit) bevat quasi nooit een interne geheugenmodule met uitzondering van cachegeheugen en registers. Het heeft daarom altijd externe componenten nodig voor opslag en I/O (input/output).



Figuur 1: Opbouw microprocessor

De microprocessor bevat een rekeneenheid, de **ALU** (Arithmetic and Logic Unit) die continu rekenkundige elementaire bewerkingen uitvoert zoals optellen, aftrekken, binaire AND, binaire OR enz. De data dient van de externe geheugenmodule te komen en het berekende resultaat moet weer hierin worden geplaatst.

(Opmerking: de geheugenmodule bevat vaak een zone om gewone **data** (variabelen) op te slaan en een aparte zone voor **programmacode**; meer info volgt verder.)

Doordat er een grote kloof is tussen de verwerkingssnelheid van de microprocessor, die vaak vele malen sneller is dan de geheugenmodule, zijn in de loop der tijd verschillende **optimalisaties** doorgevoerd. Hierdoor leest en schrijft de ALU niet rechtstreeks naar en van de geheugenmodule.

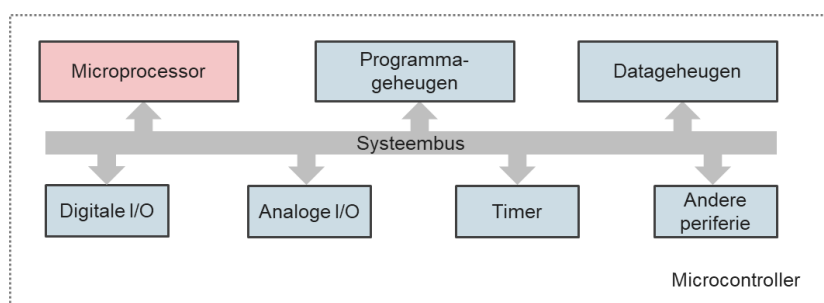
Een **eerste optimalisatie** is dat de data uit de geheugenmodule in stappen gekopieerd wordt naar de **registers** in registerbanken, die zich dicht bij de microprocessor bevinden. Deze registers fungeren als tijdelijk geheugen en zijn veel sneller dan de geheugenmodule, maar ze zijn echter beperkt in aantal en grootte. De ALU leest data uit de registerbanken, voert de benodigde bewerkingen uit en schrijft het resultaat terug naar de registerbanken. Daarna wordt het resultaat overgekopieerd naar de geheugenmodule.

Tussen de registers en de geheugenmodule bevindt zich bovendien **cachegeheugen**, een **verdere optimalisatie**. Data die vaak opgevraagd wordt uit de geheugenmodule, wordt opgeslagen in dit tussenliggende cachegeheugen. Dit geheugen is sneller dan de geheugenmodule, maar minder snel dan de registerbanken. Het cachegeheugen is groter dan de registerbanken, maar veel kleiner dan de geheugenmodule.

Enkele voorbeelden van moderne microprocessors zijn de AMD Ryzen-serie en de Intel Core-serie, die beide gebaseerd zijn op de x86-64-architectuur voor consumentencomputers, de Intel Xeon-serie voor servers, en de AMD EPYC-serie voor datacenters en bedrijfstoepassingen.

Een **microcontroller** (soms ook **MCU** genaamd, staat voor MicroController Unit) is een compacte geïntegreerde schakeling die een microprocessor, intern geheugen en extra voorzieningen (vaak **periferie** of modules genaamd). De verschillende delen zijn verbonden d.m.v. **bussen**, zie verder. Onder periferie hoort: digitale input/output, analoge input/output, timers, seriële interfaces (UART, SPI, I²C, ...).

Een aantal microcontrollers bevatten ook geavanceerdere periferie zoals USB controller, CAN bus controller, Ethernet, ...



Figuur 2: Opbouw microcontroller

Er zijn verschillende populaire microcontrollerreeksen, denk maar aan de Microchip PIC microcontrollers, Atmel AVR-reeks, ARM microcontrollers, ...

Opmerkingen:

- Tegenwoordig komt ook de term **System on Chip (SoC)** vaak voor. Hoewel een SoC, net als een MCU, alle noodzakelijke componenten op één chip bevat, zijn er toch belangrijke verschillen.

Een SoC omvat veel complexere en krachtigere componenten, zoals een Graphics Processing Unit (GPU), netwerkinterfaces (zoals WiFi en Bluetooth) en bevat vaak meerdere cores in de

microprocessor voor multitasking. Maar het is ook minder low power dan een microcontroller. Dit maakt SoC's geschikt voor toepassingen in smartphones, tablets en ook Raspberry Pi's.

- Daarnaast komt de term **embedded systeem** regelmatig voor. Het is belangrijk te begrijpen dat dit een bredere term is en geen synoniem voor een microcontroller.

Een embedded systeem verwijst naar een gespecialiseerd computersysteem dat is ontworpen om een specifieke taak of functie uit te voeren binnen een groter systeem.

Een embedded systeem kan één of meerdere microcontrollers bevatten, maar dit is geen vereiste. Het kan ook andere componenten omvatten zoals een SoC of een FPGA (Field Programmable Gate Array). In sommige gevallen bevat een embedded systeem dus helemaal zelfs geen microcontroller.

1.2 Toepassingen

Microcontrollers zijn niet meer weg te denken uit heel veel hedendaagse toestellen. Enkele voorbeelden staan hieronder.

Thuis

- Centrale verwarming en bijhorende thermostaat
- Inbraakbeveiliging
- Microgolfoven
- Speelgoed
- Digitale uurwerk
- Vaatwasser
- Audioversterker
- Domoticasystemen
- Batterijladers
- GSM
- ...



Kantoor

- Printer
- Scanner
- e-ID-lezer
- Toetsenbord
- Telefoon toestellen en -centrale
- ...

Auto

- Klimaatcontrole
- Parkeerassistentie
- Remassistentie
- GPS
- Alle systemen aangesloten op de CAN-bus
- ...



Kortom: quasi alle toestellen die wat 'intelligentie' bezitten, hebben meestal één of andere vorm van een microcontroller in zich.

1.3 Fabrikanten

Er zijn veel fabrikanten van microcontrollers, hieronder volgt een klein overzicht.

STMicroelectronics

STMicroelectronics heeft twee belangrijke reeksen microcontrollers: de STM8 (8-bit) en de STM32. De STM32-serie is gebaseerd op de ARM Cortex processor cores en heeft veel aandacht gekregen vanwege de combinatie van lage kosten en krachtige prestaties. Dit heeft de 32-bit ARM microcontrollers steeds populairder gemaakt, vooral in de industrie, waar ze vaak de standaard zijn.

Microchip

Microchip is een grote speler en is zeer populair bij de hobbyist en de industrie. Dit heeft onder andere te maken met de kostprijs van de controllers, maar ook dankzij de gratis beschikbare ontwikkelomgeving en de ruime ondersteuning via de websites van de producenten. Microchip biedt 2 grote reeksen aan: de PIC18 (8-bit) en PIC32 microcontrollerreeks. De PIC32 reeks is gebaseerd op de MIPS M-Class of M4K core.

Een andere fabrikant is **Atmel**, dat opgekocht werd door Microchip voor iets meer dan 3 miljard euro. Atmel leverde ook microcontrollers voor op Arduino-bordjes te plaatsen.

NXP

NXP is de zelfstandig geworden semiconductorafdeling van Philips en heeft nu een hele reeks microcontrollers op basis van verschillende ARM cores.

NXP heeft indertijd de andere fabrikant **Freescale** overgenomen. Freescale had microcontrollerreeksen zoals ColdFire, Power Architecture, en Kinetis (ARM-gebaseerd).

Texas Instruments

Texas Instruments biedt verschillende reeksen microcontrollers aan. Naast twee reeksen die gebouwd zijn rond ARM-processoren, heeft TI ook de MSP430 (16-bit) en C2000 (32-bit) reeksen. De MSP430 is bekend om zijn ultra-low-power eigenschappen en wordt veel gebruikt in toepassingen die energie-efficiëntie vereisen.

Renesas

Renesas is ontstaan uit een consortium van NEC, Hitachi en Mitsubishi. Renesas biedt een breed scala aan microcontrollers, waaronder ultra-low-power 8-bit en 16-bit microcontrollers (RL78, 78K, en R8C) en een 32-bit RISC (V850, SuperH) en CISC (RX) reeks.

Infineon

Infineon biedt een reeks van 8-bit (XC800), 16-bit (XC2200) en 32-bit (TriCore) microcontrollers aan. De TriCore microcontrollers zijn vooral gericht op high-performance toepassingen.

Silicon labs

Silicon Labs heeft een microcontroller-reeks gebaseerd op een gepijplijnde versie van de 8051 (8-bit) microcontroller, en biedt ook andere oplossingen voor IoT en draadloze communicatie.

Bovenstaande lijst is absoluut niet volledig en gevoelig voor verandering. Fabrikanten maken nieuwe reeksen, fusioneren, verlaten een gamma, ... De meest recente info is terug te vinden op de respectievelijke websites van de fabrikanten.

1.4 Keuze van microcontroller

Op de markt zijn er verschillende toegankelijke instapmodellen van microcontrollers met ontwikkelplatformen, zoals het bekende **Arduino**-platform.

Arduino is een open-source platform dat wordt gebruikt om eenvoudige tot geavanceerde elektronische projecten te ontwikkelen. Het bestaat uit twee hoofdcomponenten: de hardware (een Arduino-bord) en de software (de Arduino IDE). Arduino is voornamelijk ontworpen om toegankelijk te zijn voor beginners en hobbyisten.

In feite is een Arduino-bord een standaard microcontrollerbord met daarop een Atmel-microcontroller en een Arduino-bootloader. Deze bootloader is een klein stukje firmware dat ervoor zorgt dat je de microcontroller kunt programmeren via USB-interface. Hierdoor is er geen externe programmer nodig, wat het voor beginnende programmeurs en elektronicaliefhebbers eenvoudig maakt om snel hun systeem werkend te krijgen, zelfs zonder uitgebreide voorkennis.

Het meest gebruikte Arduino-bord, de Arduino Uno R3, bevat een 8-bit ATmega328P-microcontroller die werkt op een kloksnelheid van 16 MHz.



Figuur 3: Arduino Uno R3 (bron: wikipedia.org)

Voordelen van bovenstaand bord zijn:

- Prijs
- (stacking) headers voorzien zodat je snel kan prototypen
- Veel shields beschikbaar
- Programmeerbaar via USB-interface

Nadelen:

- Weinig tot geen direct beschikbare IO (knoppen en LED's). Vooraleer je kan starten met programmeren moet je dus eerst hardware koppelen.
- De print heeft een vaste afmeting die misschien absoluut niet past in het project dat je voor ogen hebt.
- De standaard IDE van Arduino is heel eenvoudig, maar mist enkele handige tools die een professionele gebruiker er graag bij heeft (vooral qua debugging) ...
- Moeilijk te gebruiken voor tijd kritische systemen (timers, vaste looptijden, ...).

Arduino is leuk, gemakkelijk, je krijgt snel 'leven' in je projecten, maar je kan het niet gebruiken als standaard om embedded systemen te leren programmeren.

Maar op een grondige manier microcontrollers leren programmeren is niet evident. Als je het echt goed wil doen, moet je tijd investeren en ervaring opdoen. Alleen op die manier kan je de gelimiteerde resources van een embedded systeem ten volle benutten.

"Using an Arduino to learn programming is like using MacDonalds to learn cooking; you get your meal, very fast, but you don't get the skills to cook yourself. When you need a quick meal, the Mac can be OK (debatable, but just to make my point), but it's not a cooking class."

(oorspronkelijke bron: <http://www.hackvandedam.nl/blog/?p=762>)

Andere vaak gehoorde negatieve bevindingen van Arduino zijn:

- Slechte IDE
- Opsplitsen van code wordt niet gestimuleerd in de IDE
- Slecht gekozen benamingen: bijvoorbeeld `analogWrite()`. Het zet eigenlijk geen analoge waarde, maar stelt een PWM-sigitaal in. Daarenboven kan je niet zomaar de PWM-frequentie aanpassen.
- Geen *exact width* variabelen (zie verder)
- De documentatie laat teveel zaken onbesproken voor de professionele gebruiker. Dat kan handig zijn voor de beginner, maar eenmaal je meer wil, kom je een 'black box' tegen.
- Door het black box-principe is het heel moeilijk om firmware te schrijven met een minimale footprint.

Wees dus op je hoede voor het Arduino-beest, het heeft zeker zijn plaats binnen de elektronica-ICT-wereld, maar niet overal! In de industrie wordt Arduino heel weinig gebruikt.

Vandaar dat we het Arduino-bordje niet zullen behandelen in het vak *Microcontrollers* en de vervolgvakken *IoT Devices* en *IoT Systems*.

In dit vak *Microcontrollers* zullen we gebruikmaken van de **STM32F091RC-microcontroller** van **STMicroelectronics**. Dit is een veelzijdige en energie-efficiënte 32-bit ARM microcontroller die geschikt is voor zowel educatieve toepassingen als industriële toepassingen.

De STM32F091RC staat bekend om zijn brede ondersteuning, uitgebreide documentatie en veelgebruikte ontwikkeltools, gecombineerd met een lage kostprijs, wat het een uitstekende keuze maakt voor het leren werken met microcontrollers.

1.5 Onderdelen

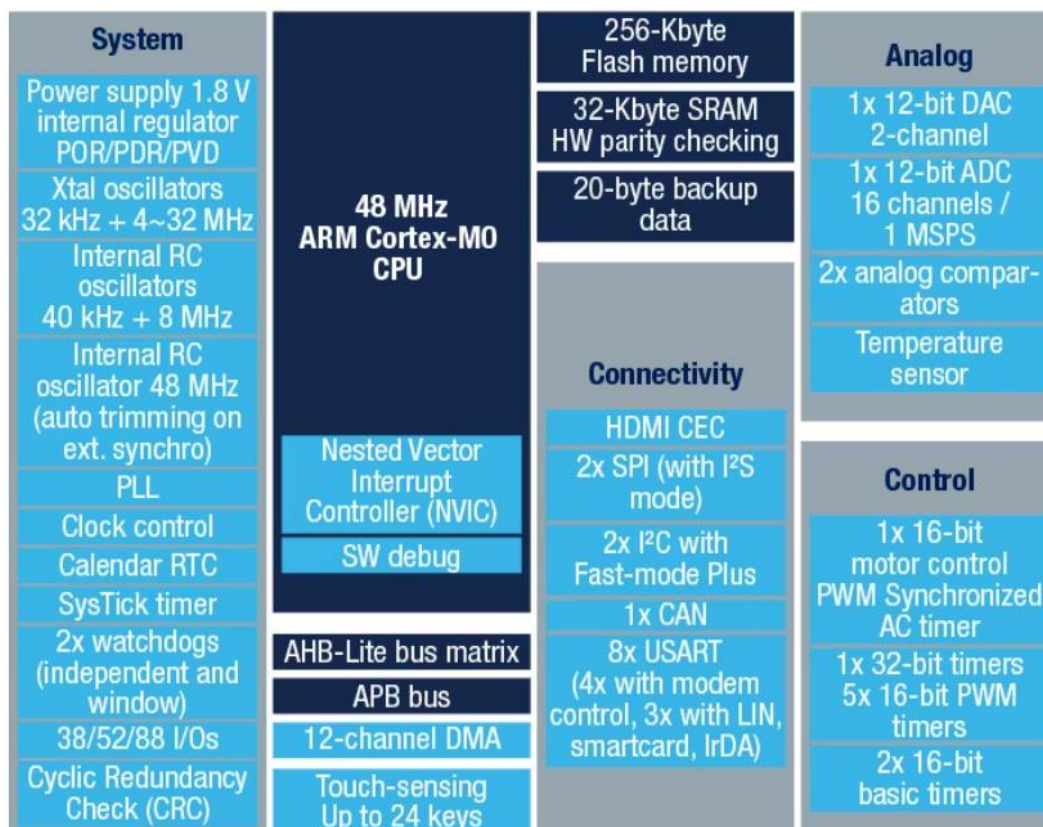
Veelvoorkomende onderdelen bij een microcontroller zijn:

- **CPU:** bevat de ALU die instructies uitvoert (logische, aritmetische, lees/schrijf acties).
- **Oscillator:** maakt met behulp van een kristal een klok waarop de logica werkt. Hier wordt dus de kloksnelheid van heel het systeem bepaald.
- **Bussen:** adres-, data- en controlebussen die de verschillende blokken verbinden in de microcontroller.
- **ROM (Read Only Memory)/flash:** geheugen dat niet volatiel is. Dit geheugen wordt gebruikt om de programmacode in op te slaan. Het programma blijft ook opgeslagen als de microcontroller zonder stroom komt te staan.
 - Opmerking: er is een klein verschil tussen ROM- en flashgeheugen. Het programma in ROM wordt tijdens de fabricage of programmering vastgelegd en kan daarna niet of nauwelijks worden gewijzigd, terwijl je bij flash meerdere malen het programma overschrijven.
- **RAM (Random Access Memory):** geheugen dat volatiel is, maar sneller dan andere types geheugen (ROM of flash).
- **Timers:** hardware modules die tellen en signalen geven wanneer een bepaalde tijd verstreken is.
- **Interrupt controller:** bundelt verschillende interrupts die al dan niet doorgegeven worden aan de processor.
- **IO,** diverse vormen:
 - Parallele IO: één byte wordt bijvoorbeeld met 8 lijnen doorgestuurd naar een ontvanger.

- Seriële IO: één byte wordt bit per bit over 1 lijn overgestuurd (bv. via UART, staat voor Universal Asynchronous Receiver-Transmitter).
- Analoge IO: het voltage toegevoerd aan een analoge input wordt door een ADC (Analog Digital Converter) omgezet tot een binaire representatie. Analoge input kan bijvoorbeeld een microfoon zijn.
Bij DAC (Digital Analog Converter) wordt vanuit een digitale waarde een voltage gegenereerd. Dit wordt bijvoorbeeld gebruikt bij het genereren van de voltages voor een luidspreker.
- **Watchdog timer:** dit is een component die af en toe een trigger moet krijgen, zo niet wordt de CPU gereset. Dit voorkomt dat de CPU, door bijvoorbeeld een bug, voor altijd in een lus blijft 'vastzitten'.

Bovenstaande lijst is niet compleet. De fabrikant van een microcontroller kan ervoor kiezen om meer of minder van dergelijke periferie te implementeren. Het is aan de ontwerper van het volledige systeem om een verstandige keuze te maken qua microcontrollertype.

Om een duidelijk overzicht te bekomen van de interne modules van onze microcontroller, bekijk je best onderstaand blokschema:



Figuur 4: overzicht van de modules in de STM32F091RC (bron: st.com)

1.6 Classificatie

Microcontrollers kunnen onder meer ingedeeld worden volgens:

- Architectuur
- Woordbreedte
- Complexiteit instructieset
- Type instructieset
- Fabrikant en model

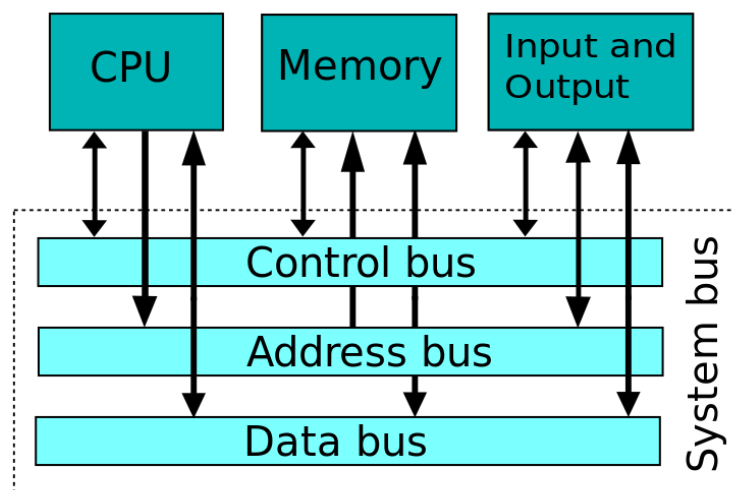
1.6.1 Architectuur

Von Neuman architectuur

Bij de Von Neumann-architectuur maakt de CPU gebruik van **één systeembus** voor zowel het datageheugen (waar de gegevens worden opgeslagen) als het programmeergeheugen (waar de code/instructies worden bewaard). Dit betekent dat zowel instructies als data over dezelfde bus moeten worden getransporteerd, wat kan leiden tot een 'bottle neck': de CPU kan niet tegelijkertijd instructies en data ophalen, wat de prestaties kan beperken.

Het grootste voordeel van de **Von Neumann-architectuur** is de eenvoud van het ontwerp, wat het aantrekkelijk maakt voor veel toepassingen. Het nadeel is echter dat de gedeelde bus voor instructies en data de snelheid kan beïnvloeden.

De meeste **microprocessoren** zijn hoofdzakelijk volgens deze architectuur gemaakt.

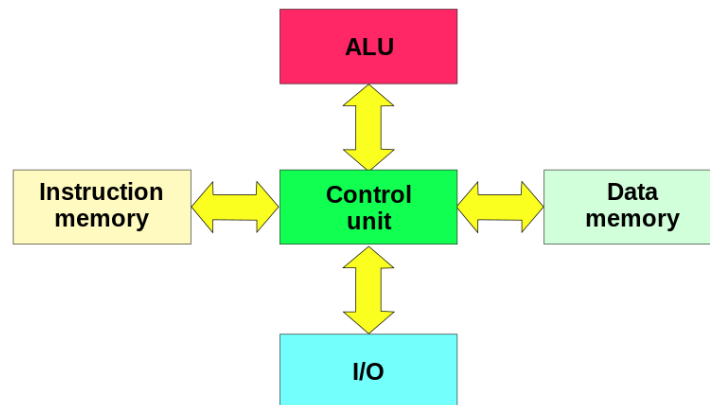


Figuur 5: Von Neuman-architectuur (bron: wikipedia.org)

Harvard architectuur

De **Harvard-architectuur** heeft **gescheiden bussen** voor het datageheugen en het programmeergeheugen, wat betekent dat de CPU gelijktijdig gegevens en instructies kan ophalen zonder te wachten op de andere bus. Dit kan de prestaties aanzienlijk verbeteren, vooral in systemen waar snelheid cruciaal is.

De meeste **microcontrollers** zijn met deze architectuur ontworpen.



Figuur 6: Harvard-architectuur (bron: wikipedia.org)

Een ander voordeel van de Harvard-architectuur is dat het datageheugen en het programmeergeheugen verschillende eigenschappen kunnen hebben, zoals verschillende timing of woordbreedte. Dit maakt het mogelijk om verschillende soorten geheugen te gebruiken voor instructies en data, zoals ROM/flash voor het programma en RAM voor de gegevens.

Bij de **gewijzigde Harvard-architectuur** (Modified Harvard Architecture) zijn de twee geheugens, hoewel gescheiden, na de cache niet volledig geïsoleerd van elkaar. Moderne microprocessors gebruiken vaak een gescheiden cache voor zowel instructie- als datageheugen, maar na de cache worden de bussen weer samengevoegd op één bus voor verdere communicatie met de CPU. Dit biedt een balans tussen de voordelen van gescheiden geheugens en de kosten van een volledig gescheiden systeem.

In sommige gevallen, zoals bij bepaalde microcontrollers, wordt er gedeeltelijk gebruik gemaakt van de Harvard-architectuur. Bijvoorbeeld, bij de PIC18-reeks blijven strings van het programma opgeslagen in ROM in plaats van te worden gekopieerd naar RAM, en kunnen ze via een speciale pointer in de C-taal direct worden benaderd. Dit bespaart geheugen en maakt efficiënter gebruik van de beschikbare resources.

1.6.2 Woordbreedte

De woordbreedte bepaalt op hoeveel data de processor per klokpuls bewerkingen uitvoert.

Microprocessors in laptops en computers zijn tegenwoordig bijna allemaal 64-bit. Zelfs voor de kleinste pc's wordt veelal een 64-bit processor gebruikt.

Er zijn momenteel nog veel 8-bit en 16-bit microcontrollers in gebruik. Er is wel duidelijk een trend naar 32-bit microcontrollers. De prijs van dergelijke 32-bitters is de laatste jaren fel gedaald, waardoor het ook in goedkopere systemen mogelijk wordt om een 32-bit microcontroller te gebruiken.

In de praktijk worden de meeste microcontrollers geprogrammeerd in de programmeertaal C. In deze programmeertaal hebben de **integer types niet altijd dezelfde grootte (breedte)**. Dit stelt natuurlijk problemen bij gebruik van deze integer types. Om deze problemen te vermijden definieert men types die onafhankelijk van de woordbreedte dezelfde grootte hebben.

In ons geval voorzien we deze types:

```
/* exact-width signed integer types */
typedef signed char int8_t;
typedef signed short int int16_t;
typedef signed int int32_t;
typedef signed __INT64 int64_t;

/* exact-width unsigned integer types */
typedef unsigned char uint8_t;
typedef unsigned short int uint16_t;
typedef unsigned int uint32_t;
typedef unsigned __INT64 uint64_t;
```

Figuur 7: exact-width integers (stdint.h)

Als we dus bijvoorbeeld een getal willen voorstellen dat als bereik 0 tot 54205 moet hebben en toch zo efficiënt mogelijk gebruik moet maken van de beschikbare resources, kunnen we het best het type `uint16_t` gebruiken.

Bekijk zelf eens enkele andere mogelijkheden.

1.6.3 Complexiteit instructieset

CISC (Complex Instruction Set Computer)

CISC-processoren zijn ontworpen om complexe taken met één enkele instructie uit te voeren. Een CISC-instructie kan meerdere acties omvatten, zoals het lezen van een waarde uit het geheugen, het uitvoeren van een bewerking en het terugschrijven van het resultaat naar het geheugen.

Bijvoorbeeld:

- Voorbeeldinstructie: bereken de som van twee getallen uit het geheugen en sla het resultaat direct op in een register of geheugenlocatie.

CISC-architecturen hebben doorgaans een grote en diverse instructieset, wat de hardwarecomplexiteit verhoogt. Dit maakt ze geschikt voor systemen waar compatibiliteit met bestaande software belangrijk is, zoals bij de Intel x86-architectuur, die veel wordt gebruikt in desktops en servers.

RISC (Reduced Instruction Set Computer)

RISC-processoren zijn ontworpen met eenvoud en efficiëntie in gedachten. In tegenstelling tot CISC zijn de individuele instructies in een RISC-architectuur zeer eenvoudig. Deze eenvoud maakt het mogelijk om een hoge kloksnelheid te bereiken, omdat elke instructie in één enkele kloksnelheidscyclus kan worden uitgevoerd.

De term "reduced" verwijst niet naar het aantal instructies in de set, maar naar de hoeveelheid werk die per instructie wordt uitgevoerd.

Bijvoorbeeld:

- Voorbeeldinstructies:
 1. Lees een eerste waarde uit het geheugen naar een register.
 2. Lees een tweede waarde uit het geheugen naar een register.
 3. Bereken de som van de twee waarden in de registers en schrijf het resultaat weg naar een register.
 4. Schrijf het resultaat naar het geheugen.

Een voorbeeld van een veelgebruikte RISC-architectuur is ARM, die niet alleen in onze STM-microcontroller voorkomt, maar ook in tablets en smartphones.

1.6.4 Type instructieset

De bekendste instructieset is wellicht de **x86**. Dit is een instructieset die ontwikkeld werd voor het gebruik op de Intel 8086 processoren. Oorspronkelijk was dit een 16-bit instructieset. Later volgden 32- en 64-bit versies.

Andere instructiesets zijn PowerPC, SPARC, PIC, ARM.

Veelal is een instructieset gekoppeld met een fabrikant. Echter wordt de x86 niet enkel door Intel, maar ook door meerdere fabrikanten gebruikt waaronder AMD.

Van de **ARM instructieset** zijn er ook meerdere fabrikanten. De firma ARM (www.arm.com, weetje: ARM staat trouwens voor Advanced RISC Machines) produceert zelf geen microprocessoren maar verkoopt ontwerpen aan andere firma's zoals NXP en Qualcomm.

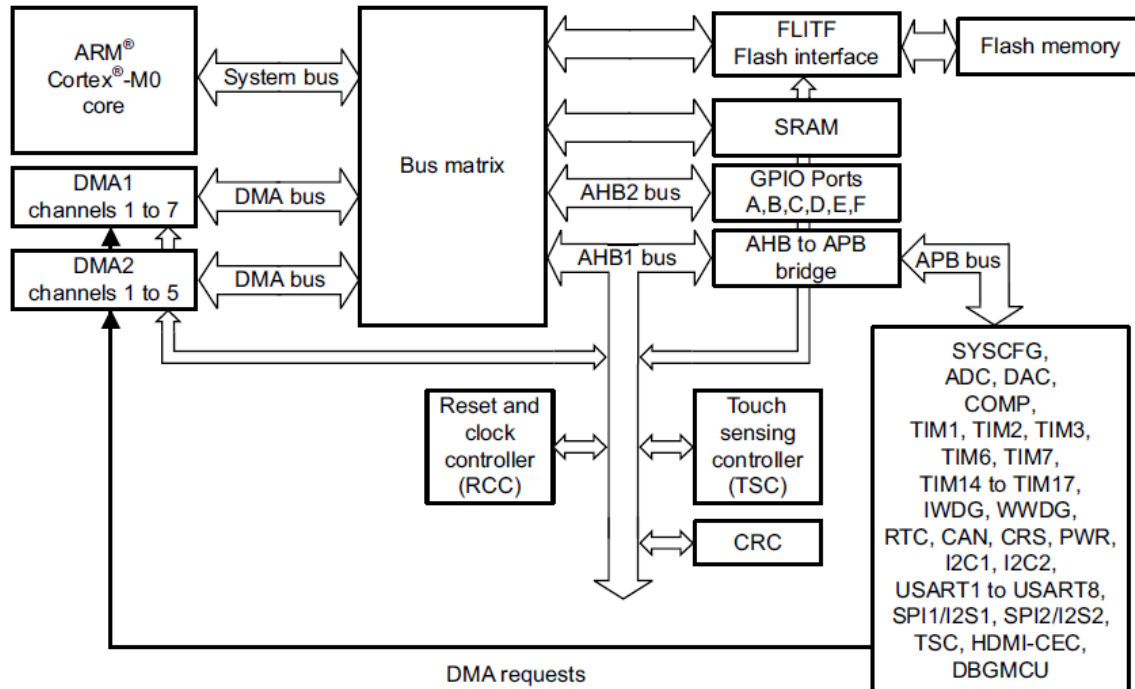
Ook van onze microcontroller STM32F091RC is de architectuur ontworpen door het bedrijf ARM, maar in een chip geplaatst door de fabrikant ST (www.st.com).



2 STM32-architectuur (STM32F091RC Cortex M0)

2.1 Blokschema

Onze **STM32F091RC ARM microcontroller** heeft onderstaande architectuuropbouw:



Figuur 8: STM32F0-architectuur (bron: STM32F0 Reference Manual - DM00031936)

Merk op dat het niet meer zo eenvoudig is om hier nog de opsplitsing te maken tussen Von Neuman en Harvard. De ARM-architecturen zijn eigenlijk specifiek ontworpen met meestal één bepaalde focus voor ogen.

In ons geval is dit bij de Cortex M0-reeks: *"The ARM® Cortex®-M0 processor is the smallest ARM processor available. The exceptionally small silicon area, low power and minimal code footprint of the processor enables developers to achieve 32-bit performance at an 8-bit price point, bypassing the step to 16-bit devices. The ultra-low gate count of the Cortex-M0 processor also enables its deployment in analog and mixed signal devices."*

(bron: <http://www.arm.com/products/processors/cortex-m/cortex-m0.php>)

2.2 Beschrijving

Bovenstaand blokschema kan logisch ingedeeld worden in Masters, System bus, Slaves en AHB naar APB brug.

Masters

Bovenstaand blokschema heeft faciliteiten voor drie masters. Een master speelt baas in een elektronisch systeem. Andere componenten uit het zelfde systeem moeten gehoorzamen/luisteren naar wat de master hen vraagt. De **drie masters** hier zijn: **ARM Cortex M0 core**, **DMA1** en **DMA2**.

In de STM32F091RC is de ARM Cortex M0 core hét hart van het systeem (de belangrijkste master). Dit is de CPU waar alle berekeningen plaatsvinden. Een Cortex M0 core is een 'entry level' ARM microcontroller. Dat wil zeggen dat het een van de eenvoudigste ARM Cortex cores is die er bestaan. De complexere Cortex cores worden bijvoorbeeld ingezet bij smartphones maar vallen buiten het bestek van deze cursus.

De DMA1 en DMA2 module zorgen voor Direct Memory Access faciliteiten. DMA-gebruik is echter niet voor beginners, daarom volgt meer uitleg in een vervolgcursus.

System bus

De systeembus uit voorgaand blokschema verbindt de CPU met de Bus Matrix. Die Bus Matrix regelt de arbitrage tussen de core en de DMA-bussen, m.a.w. het zorgt dat er geen bits botsen en dergelijke. Het is dus die Bus Matrix die bij een DMA request de nodige stappen zal ondernemen om de CPU te ontheven van zijn heel repetitieve tijdrovende acties en zo de efficiëntie van heel het systeem omhoog te krikken door via DMA-technieken bepaalde acties te voltooien.

Slaves

Als er masters zijn, moeten er ook slaves zijn. In de Cortex M0 core zijn dit er vier:

- **FLITF:** FLash memory InTerFace
- **SRAM:** Static RAM
- **AHB1** (Advanced High-performance Bus 1) met "**AHB naar APB-brug**"
 - APB staat voor Advanced Peripheral Bus
- **AHB2:** Advanced High-performance Bus 2

AHB naar APB brug

De AHB1, bezit een brug (bridge) naar de APB, deze laatste maakt connectie met Timers, UART's, RTC, I²C, ... Doorheen die brug kan de periferie de CPU bereiken. Dit wordt verder in de cursus heel belangrijk.